# Thoura AI:

# Construction Materials & Services Marketplace Requirements

## 1. Executive Summary

Constructs Hub (BanaBuild) is a comprehensive e-commerce platform specializing in construction materials and services, connecting homeowners with merchants, technicians, and delivery services. The platform provides end-to-end solutions from material purchase to professional installation and delivery.

## 2. Platform Architecture

### 2.1 Technical Stack

- **Frontend**: React.js (Web), React Native (Mobile - iOS/Android/Harmony OS)
- **Backend**: Node.js with Express.js
- **Database**: MongoDB (documents) + PostgreSQL (transactions)
- **Real-time**: Socket.IO for chat notifications
- **Maps**: Google Maps API with location services
- **Payment**: Stripe integration for credit cards, PayPal alternative option
- **Push Notifications**: Firebase Cloud Messaging
- **File Storage**: AWS S3 for product images and documents

### 2.2 System Architecture

- **Microservices**: Authentication, Products, Orders, Payments, Users, Notifications
- **Load Balancing**: Nginx reverse proxy
- **Caching**: Redis for session management and frequently accessed data
- **Database**: Primary-Replica setup for scalability

# 3. User Types and Personas

## 3.1 Homeowner/Normal User

- **Profile**: Personal information, saved addresses (with Google Maps integration), payment methods
- **Capabilities**: Browse products, search, purchase, hire technicians, track deliveries, leave reviews

## 3.2 Merchant/Store

- **Profile**: Business information, inventory management, store location
- **Capabilities**: Manage products, process orders, handle pickups, manage technician partnerships

## 3.3 Technician/Worker

- **Profile**: Skills, experience, hourly rates, availability zones
- **Capabilities**: Receive job requests, communicate with clients, create work plans, track earnings

## 3.4 Driver/Vehicle Owner

- **Profile**: Vehicle specifications, availability, delivery zones
- **Capabilities**: Accept delivery jobs, track routes, update delivery status

## 3.5 Administrator

- **Capabilities**: Platform management, user moderation, dispute resolution, analytics

# 4. Core Features

## 4.1 Search and Discovery System

**Web/Mobile Search Interface**:

- Search bar with autocomplete functionality

- Keyword-based search (wash sink, Italian sink, paint types, etc.)
- Advanced filters: price range, ratings, location, availability
- Search result grid layout with product cards

**Product Cards Display**:

- High-quality product images
- Product name and brief description
- Price and vendor/store information
- Review score (1-5 stars)
- Quick actions: "Buy Now", "Add to Cart"
- Additional icons: Store location, Delivery/Pickup toggle, Technician availability, Stock status
- "Get Quote" button for custom pricing

## 4.2 User Registration and Profiles

**Registration Process**:

- Multi-step registration with email verification
- Profile setup with address mapping via Google Maps drag-and-drop
- Payment method integration (secure credit card storage)

**Profile Management**:

- Personal/business information
- Saved addresses with location markers
- Order history and preferences
- Saved payment methods
- Notification settings

## 4.3 Shopping Cart and Checkout

**Shopping Cart**:

- Add/remove items with quantity adjustment
- Real-time price calculation
- Delivery option selection
- Technician service integration

- Persistent cart across sessions

**Checkout Process**:

- Cart review with item details
- Delivery method selection (pickup/delivery)
- Technician service selection (if applicable)
- Payment method selection
- Order confirmation and redirect to review page

## 4.4 Technician Matching System

**Technician Directory**:

- Grid layout of available technicians
- Filter options: cost, hourly rate, availability, location, review score
- Distance calculation and travel time estimation
- Partnership highlighting (store-registered technicians)

**Technician Profile Cards**:

- Profile photo and basic information
- Statistics: completed jobs, satisfaction rating
- Hourly rate and service cost
- Availability status
- Location and travel time
- "Hire Now" and "Get Quote" buttons

**Job Request System**:

- Automated notification to technicians
- Pre-formatted job description with material specifications
- Direct chat integration for client-technician communication
- Work plan and cost proposal creation
- Client approval/negotiation workflow

## 4.5 Delivery and Vehicle Management

**Vehicle Requirements Calculation**:

- Automatic vehicle type recommendation based on order weight/volume
- Weight categories: Small (motorcycle/car), Medium (van), Large (truck)
- Volume calculation for bulky items
- Cost estimation based on distance and vehicle type

**Delivery Tracking**:

- Real-time GPS tracking integration
- Google Maps-style interface with driver location
- Estimated time of arrival updates
- Delivery status notifications
- Driver rating system

## 4.6 Payment Processing

**Payment Methods**:

- Direct site payment (merchant account)
- Credit card processing (Stripe integration)
- Saved payment methods for future purchases
- Split payment processing for multiple services

**Payment Flow**:

- Material cost → Merchant account
- Service cost → Technician account
- Delivery cost → Driver account
- Transaction history in user dashboards
- Refund and dispute resolution system

## 4.7 Review and Rating System

**Product Reviews**:

- Star rating (1-5)
- Written review with optional photos
- Verification of purchase requirement
- Review moderation for inappropriate content

**Service Reviews**:

- Technician performance evaluation
- Work quality and professionalism ratings
- Communication and timeliness feedback
- Overall satisfaction scoring

**Delivery Reviews**:

- Driver performance evaluation
- Delivery time and condition ratings
- Professionalism and communication feedback

# 5. Dashboard Requirements

## 5.1 Homeowner Dashboard

**Overview**:

- Active orders tracking
- Upcoming appointments
- Recent activity feed
- Quick actions (search, new order)

**Order Management**:

- Order history with status tracking
- Delivery tracking interface
- Service appointment scheduling
- Payment history and receipts

**Reviews Section**:

- Pending review notifications
- Review history
- Ability to modify existing reviews

## 5.2 Merchant Dashboard

**Order Management**:

- Incoming orders notifications
- Order processing workflow
- Pickup scheduling
- Inventory management integration

**Analytics Screen**:

- Sales performance metrics
- Customer satisfaction trends
- Popular products analysis
- Revenue breakdown by category

**Review Management**:

- Customer reviews monitoring
- Response capability for reviews
- Rating trends analysis
- Customer satisfaction insights

## 5.3 Technician Dashboard

**Job Management**:

- Incoming job requests
- Accepted jobs scheduling
- Work plan creation tools
- Time tracking integration

**Performance Analytics**:

- Job completion statistics
- Customer satisfaction ratings
- Earnings breakdown
- Availability optimization

**Communication Hub**:

- Client messaging interface
- Job proposal management
- Schedule coordination tools

## 5.4 Driver Dashboard

**Job Management**:

- Delivery job assignments
- Route optimization tools
- Delivery status updates
- Pickup confirmation system

**Performance Tracking**:

- Delivery statistics
- Customer ratings
- Earnings summary
- Availability management

## 5.5 Admin Dashboard

**Platform Management**:

- User account management
- Content moderation
- Dispute resolution tools
- System monitoring

**Analytics and Reporting**:

- Platform-wide metrics
- Revenue tracking
- User growth analysis
- Performance indicators

# 6. Technical Requirements

## 6.1 Performance Requirements

- **Response Time**: <2 seconds for all page loads
- **Search Performance**: <1 second for search queries
- **Real-time Updates**: <500ms for notification delivery
- **Uptime**: 99.9% availability
- **Concurrent Users**: Support 10,000+ concurrent users

## 6.2 Security Requirements

- **Data Encryption**: AES-256 for sensitive data
- **Payment Security**: PCI DSS compliance
- **Authentication**: JWT-based secure authentication
- **Data Protection**: GDPR compliance for user data
- **Regular Security Audits**: Quarterly penetration testing

## 6.3 Integration Requirements

- **Payment Gateways**: Stripe, PayPal integration
- **Maps**: Google Maps API with routing
- **Push Notifications**: Firebase Cloud Messaging
- **Email Service**: SMTP integration for notifications
- **File Storage**: AWS S3 integration
- **Analytics**: Google Analytics integration

## 6.4 Mobile App Requirements

- **Cross-Platform**: iOS, Android, Harmony OS support
- **Offline Mode**: Basic functionality without internet
- **Push Notifications**: Real-time updates
- **Location Services**: GPS tracking for delivery
- **Camera Integration**: Product photography and document scanning

# 7. Development Phases

## Phase 1: Core Platform (Months 1-3)

- User authentication and profiles
- Product catalog and search
- Shopping cart and basic checkout
- Payment processing integration

## Phase 2: Services Integration (Months 4-6)

- Technician matching system
- Job request and communication
- Work plan creation tools
- Service scheduling

## Phase 3: Delivery System (Months 7-9)

- Vehicle management system
- Delivery tracking integration
- Route optimization
- Driver management

## Phase 4: Advanced Features (Months 10-12)

- Review and rating system
- Analytics dashboards
- Mobile app development
- Performance optimization

# 8. Success Metrics

- **User Acquisition**: 50,000+ registered users in first year
- **Transaction Volume**: 10,000+ monthly transactions
- **Service Matching**: 90% job completion rate
- **Customer Satisfaction**: 4.5+ average rating
- **Platform Revenue**: 15% commission on all transactions

Gemini AI

This is a comprehensive and ambitious project. As a full-stack engineer and platform owner, I have synthesized your vision into a structured **Product Requirements Document (PRD)**. This covers the functional requirements, user experience design, and the logic for your construction ecosystem platform, **BanaBuild**.

## 1. System Architecture & Tech Stack

To support the web-based store and mobile apps (iOS, Android, HarmonyOS) while maintaining a single backend:

- **Frontend:** React Native with **Expo** (allows code sharing between iOS, Android, and Web/HarmonyOS targets).
- **Backend: NestJS** (Node.js) for a scalable, modular microservices architecture.
- **Database:** PostgreSQL (for relational data like transactions and user profiles) + Redis (for real-time tracking/notifications).
- **Mapping:** Google Maps Platform API for Geofencing and "Drop-bin" location services.
- **Cloud Hosting:** Render or Railway for the backend; Firebase for push notifications and real-time chat.

# 2. Core User Flows & Functional Requirements

## A. The Search & Discovery Engine

The system must handle a wide variety of construction-specific metadata.

- **Smart Search:** Implementation of keyword-based filtering (e.g., "Oily indoor paint") with a **Matrix Grid View**.
- **Grid Item Components:**
    - Image, Name, Vendor Name, Price.
    - Review Score (Star rating).
    - **Action Icons:** "Buy Now," "Add to Basket," "Store Location (Map Link)."
    - **Service Toggles:** "Delivery vs. Pick-up" and "Installation Service Needed?"
    - **Availability Status:** Dynamic label (e.g., "In Stock" or "Available in 5 Days").
    - **Quotation Trigger:** If price == 0 or null, replace "Buy Now" with "Get Quotation."

## B. The Technician (Service) Marketplace

Triggered when the user selects "I need a technician."

- **Service Matching:** Filters workers based on the product category (e.g., buying a sink triggers "Plumber" filter).
- **Technician Profile Card:**
    - Stats: Completed jobs, Hourly rate/Estimated cost.
    - Real-time Location: ETA to the user's dropped pin.
    - **Hire Now / Get Quote:** Triggers the Correspondence System.
- **Negotiation Workflow:** * **Auto-Letter Generation:** Backend populates a template: *"Client [Name] purchased [Material Specs]. Requesting: [Fitting/Assembly]."*
    - **Phase-Based Planning:** Worker submits a plan (Phase 1: Prep, Phase 2: Install) with costs per phase.
    - **Status Logic:** Draft -> Negotiation -> Agreed -> Active -> Completed.

## C. Logistics & Intelligent Delivery Calculation

The system must differentiate between a "Parcel" and "Freight."

- **Load Calculation Logic:**

$$\text{Total Weight} = \sum (\text{Item Weight} \times \text{Quantity})$$

- **Vehicle Matching:**
    - **Small (< 50kg):** Motorbike / Small Car.
    - **Medium (50kg - 500kg):** Pickup Truck.
    - **Heavy (> 500kg):** 3-ton to 10-ton Trucks / Rebar Trailers.
- **Tracking:** Real-time GPS integration using a driver-side app (similar to Uber/Kareem).

# 3. UI/UX: AI-Generated Page Descriptions

## The Review & Feedback Page (User Side)

This page appears post-purchase to ensure ecosystem health.

- **Layout:** A clean, vertical scrollable list divided into three cards: **Product, Store, and (if applicable) Driver/Technician.**
- **Visual Elements:** * **Star Slider:** A 1-5 slider that changes color from Red to Gold.
    - **Media Upload:** A "drag and drop" zone to upload photos of the finished work or delivered material.
    - **Quick Tags:** Buttons like "Prompt Delivery," "Professional Work," or "Exactly as Described."
- **Final Action:** A "Submit All" button that triggers the release of escrow funds if applicable.

## The Merchant/Provider Dashboard (Internal)

- **Analytics Overview:** High-level cards showing **Total Sales, Active Jobs, and Reputation Score.**
- **User Honesty Evaluation:** A private rating system where the Merchant rates the Buyer (e.g., "Clear communication," "On-time for pickup").
- **Financial Ledger:** A table showing:
    - Gross Amount | Platform Fee | Net Transfer | Status (Pending/Transferred)
- **AI Insights:** A text block stating: *"Users in Dammam are currently searching for 20% more 'Waterproofing' materials than last month. Consider restocking."*

# 4. Payment & Financial Split Logic

The "Split-Payment" system is critical. The user pays **one total sum**, but the API handles the distribution.

| Recipient | Triggers Payment | Calculation |
|---|---|---|
| **Merchant** | Product Checkout / Pickup | Item Price - Platform Commission |
| **Technician** | Job Completion Approval | Agreed Quote + Extra Materials |
| **Logistics** | Delivery Confirmation | Distance-based fee + Weight Surcharge |

**Payment Gateway Flow:**

1. User enters Credit Card (Option to **Save Card** using Tokenization for PCI compliance).
2. Payment successful -> Funds held in a virtual "BanaBuild Escrow."
3. On milestone completion (Pickup/Delivery/Fixing) -> Funds pushed to respective bank accounts.

# 5. Summary of Dashboards (The 4 User Types)

1. **Consumer App:** Focused on search, tracking, and managing "My Home" repairs.
2. **Merchant Web Portal:** Inventory management, order fulfillment, and buyer ratings.
3. **Technician Mobile App:** Lead management, chat, scheduling, and job history.
4. **Driver Mobile App:** Route optimization, weight/load capacity management, and tracking.

### Next Technical Step

To move forward, we should define the **Database Schema** for the "Job Phases" and the **API endpoints** for the heavy-load delivery calculator. Would you like me to draft the JSON structure for the Technician's "Work Plan" or the Merchant's inventory database?

GPT 4.5 deep thinking

This is a comprehensive and ambitious project. As a full-stack engineer and platform owner, I have synthesized your vision into a structured **Product Requirements Document (PRD)**. This covers the functional requirements, user experience design, and the logic for your construction ecosystem platform, **BanaBuild**.

## 1. System Architecture & Tech Stack

To support the web-based store and mobile apps (iOS, Android, HarmonyOS) while maintaining a single backend:

- **Frontend:** React Native with **Expo** (allows code sharing between iOS, Android, and Web/HarmonyOS targets).
- **Backend: NestJS** (Node.js) for a scalable, modular microservices architecture.
- **Database:** PostgreSQL (for relational data like transactions and user profiles) + Redis (for real-time tracking/notifications).
- **Mapping:** Google Maps Platform API for Geofencing and "Drop-bin" location services.
- **Cloud Hosting:** Render or Railway for the backend; Firebase for push notifications and real-time chat.

# 2. Core User Flows & Functional Requirements

## A. The Search & Discovery Engine

The system must handle a wide variety of construction-specific metadata.

- **Smart Search:** Implementation of keyword-based filtering (e.g., "Oily indoor paint") with a **Matrix Grid View**.
- **Grid Item Components:**
    - Image, Name, Vendor Name, Price.
    - Review Score (Star rating).
    - **Action Icons:** "Buy Now," "Add to Basket," "Store Location (Map Link)."
    - **Service Toggles:** "Delivery vs. Pick-up" and "Installation Service Needed?"
    - **Availability Status:** Dynamic label (e.g., "In Stock" or "Available in 5 Days").
    - **Quotation Trigger:** If price == 0 or null, replace "Buy Now" with "Get Quotation."

## B. The Technician (Service) Marketplace

Triggered when the user selects "I need a technician."

- **Service Matching:** Filters workers based on the product category (e.g., buying a sink triggers "Plumber" filter).
- **Technician Profile Card:**
    - Stats: Completed jobs, Hourly rate/Estimated cost.
    - Real-time Location: ETA to the user's dropped pin.
    - **Hire Now / Get Quote:** Triggers the Correspondence System.
- **Negotiation Workflow:** * **Auto-Letter Generation:** Backend populates a template: *"Client [Name] purchased [Material Specs]. Requesting: [Fitting/Assembly]."*
    - **Phase-Based Planning:** Worker submits a plan (Phase 1: Prep, Phase 2: Install) with costs per phase.
    - **Status Logic:** Draft -> Negotiation -> Agreed -> Active -> Completed.

## C. Logistics & Intelligent Delivery Calculation

The system must differentiate between a "Parcel" and "Freight."

- **Load Calculation Logic:**

$$\text{Total Weight} = \sum (\text{Item Weight} \times \text{Quantity})$$

- **Vehicle Matching:**
    - **Small (< 50kg):** Motorbike / Small Car.
    - **Medium (50kg - 500kg):** Pickup Truck.
    - **Heavy (> 500kg):** 3-ton to 10-ton Trucks / Rebar Trailers.
- **Tracking:** Real-time GPS integration using a driver-side app (similar to Uber/Kareem).

# 3. UI/UX: AI-Generated Page Descriptions

## The Review & Feedback Page (User Side)

This page appears post-purchase to ensure ecosystem health.

- **Layout:** A clean, vertical scrollable list divided into three cards: **Product, Store, and (if applicable) Driver/Technician.**
- **Visual Elements:** * **Star Slider:** A 1-5 slider that changes color from Red to Gold.
    - **Media Upload:** A "drag and drop" zone to upload photos of the finished work or delivered material.
    - **Quick Tags:** Buttons like "Prompt Delivery," "Professional Work," or "Exactly as Described."
- **Final Action:** A "Submit All" button that triggers the release of escrow funds if applicable.

## The Merchant/Provider Dashboard (Internal)

- **Analytics Overview:** High-level cards showing **Total Sales, Active Jobs, and Reputation Score.**
- **User Honesty Evaluation:** A private rating system where the Merchant rates the Buyer (e.g., "Clear communication," "On-time for pickup").
- **Financial Ledger:** A table showing:
    - Gross Amount | Platform Fee | Net Transfer | Status (Pending/Transferred)
- **AI Insights:** A text block stating: *"Users in Dammam are currently searching for 20% more 'Waterproofing' materials than last month. Consider restocking."*

# 4. Payment & Financial Split Logic

The "Split-Payment" system is critical. The user pays **one total sum**, but the API handles the distribution.

| Recipient | Triggers Payment | Calculation |
|---|---|---|
| **Merchant** | Product Checkout / Pickup | Item Price - Platform Commission |
| **Technician** | Job Completion Approval | Agreed Quote + Extra Materials |
| **Logistics** | Delivery Confirmation | Distance-based fee + Weight Surcharge |

**Payment Gateway Flow:**

1. User enters Credit Card (Option to **Save Card** using Tokenization for PCI compliance).
2. Payment successful -> Funds held in a virtual "BanaBuild Escrow."
3. On milestone completion (Pickup/Delivery/Fixing) -> Funds pushed to respective bank accounts.

# 5. Summary of Dashboards (The 4 User Types)

1. **Consumer App:** Focused on search, tracking, and managing "My Home" repairs.
2. **Merchant Web Portal:** Inventory management, order fulfillment, and buyer ratings.
3. **Technician Mobile App:** Lead management, chat, scheduling, and job history.
4. **Driver Mobile App:** Route optimization, weight/load capacity management, and tracking.

### Next Technical Step

To move forward, we should define the **Database Schema** for the "Job Phases" and the **API endpoints** for the heavy-load delivery calculator. Would you like me to draft the JSON structure for the Technician's "Work Plan" or the Merchant's inventory database?

Perplixity general AI :

Below is a full product requirements specification for your platform concept: a multi-sided construction marketplace that sells materials and also coordinates technicians and delivery providers across web and mobile. The correct product framing is not a “normal online store,” but a unified construction commerce platform with four principal actors: customer, merchant, technician, and delivery provider, with marketplace-controlled payment orchestration and role-specific dashboards. This architecture is justified because construction material commerce requires advanced search, real-time

# inventory, bulk logistics, pickup and delivery choices, quotation workflows, and split payouts across multiple parties.coaxsoft+2

## Product scope

The platform should be defined as a multi-vendor construction ecosystem marketplace for retail and light professional users who need to discover, purchase, transport, install, and review construction-related goods and services in one flow. The platform must support product discovery, merchant comparison, technician hiring, delivery selection, quotations, negotiations, checkout, payment, tracking, reviews, dispute handling, and post-transaction settlement to each beneficiary account.sharetribe+1

A suitable product brand can be “Constructs Hub” or “BanaBuild,” but at requirements level the system should be modeled as a multi-tenant commerce and services platform rather than a single-store storefront. This is necessary because the scenario explicitly requires independent merchants, independent workers, independent vehicle owners/drivers, and separate transaction histories and analytics for each role.synder+2

## Business model

The core business model is a marketplace commission model with optional service fees, delivery coordination fees, and premium merchant or technician subscriptions in later phases. The platform may collect one customer payment while allocating the proceeds across merchant, technician, and delivery provider accounts through split payment logic supported by marketplace payment systems such as Stripe Connect.stripe+2

The economic logic should support three transaction categories: material sale, labor/service sale, and logistics sale. Each category must be independently priced, independently refundable where needed, and independently settled into the beneficiary’s bank account, while still appearing to the customer as one coordinated project transaction.sharetribe+1

## Actors and roles

The system must support at least the following user types:

| Role | Purpose |
|---|---|
| Customer | Searches, buys, requests quotes, hires technician, books delivery, pays, tracks, reviews. |
| Merchant / Store | Lists products, manages inventory, prices, quotations, order fulfillment, pickup readiness, delivery preparation, customer reviews, settlement history. |
| Technician / Worker | Creates service profile, receives quote requests, chats with client, submits schedule/phases/cost, gets hired, performs work, closes milestones, receives payout, reviews customer. |
| Delivery provider / Driver / Vehicle owner | Accepts delivery jobs, exposes vehicle capacity and availability, performs delivery, shares live location, completes delivery, receives payout, reviews customer. |
| Platform admin | Manages users, disputes, commissions, payouts, fraud controls, moderation, taxonomy, analytics, and compliance workflows. |

This actor model directly follows marketplace best practices that emphasize distinct journey mapping, onboarding, quality control, and operational workflows for each participant type.coaxsoft+1

## Supported platforms

The solution must be designed as one platform ecosystem with:

- Responsive web application for customers, merchants, technicians, drivers, and admins.
- Native or cross-platform mobile apps for iOS and Android.
- Future-ready architecture for HarmonyOS and similar operating systems through shared API and shared business logic layers.
- Common backend services and role-based APIs for all clients.

The front-end product strategy should use a single design system and compact web-app UI patterns because e-commerce and dashboard interfaces benefit from dense, clear, and role-aware navigation rather than marketing-style layouts.coaxsoft

## Product vision

The platform's mission is to let a non-expert house owner solve a real construction or maintenance problem end-to-end: identify the needed material, compare vendors, decide on pickup or delivery, optionally hire a specialist, negotiate the work scope if needed, pay through one platform checkout, track the order and job, and review all parties involved.

The platform should therefore optimize for problem-solving journeys, not only catalog browsing. In practice, "I have a leaking sink" or "I need indoor wall paint" should become structured demand flows that connect product, service, and logistics. This approach is aligned with construction e-commerce guidance that stresses advanced search, delivery methods, real-time inventory, and specialized operational support rather than generic retail flows.coaxsoft

## Core use cases

## Home repair

A customer notices a bathroom sink leak, searches for "wash sink," "Italian sink," "Grohe sink," or related plumbing materials, views relevant products in a result grid, compares vendors, checks stock and technician availability, adds a selected item to basket, requests a plumber, receives offers or quotations, completes payment, tracks material delivery or pickup, monitors service completion, and then reviews the merchant, technician, and possibly driver.coaxsoft

## Painting project

A customer searches for "paint," "indoor wall paint," or "oily indoor paint beige," filters by color, finish, brand, store distance, technician availability, and delivery time, purchases the paint, optionally hires a painter, receives a phased work plan, accepts or negotiates pricing, completes checkout, tracks delivery if applicable, and closes the job with review and payment settlement.coaxsoft

## Bulk materials logistics

A customer purchases heavy materials such as cement or steel rebars in tonnage quantities, and the system computes delivery vehicle requirements based on weight, volume, and store-defined transport rules. The customer then chooses among suitable transport providers by vehicle type, cost, location, and ETA before proceeding to a unified payment flow. Construction marketplaces specifically require multiple delivery methods, zone-based pricing, and specialized freight handling for oversized or bulk materials.coaxsoft

## Functional modules

## Customer-facing modules

The customer application must include:

- Registration, login, identity, and profile management.
- Saved addresses and map-selected locations.
- Search, browse, and category navigation.
- Product details and product comparison.
- Cart and checkout.
- Pickup or delivery selection.
- Technician discovery and hiring.
- Delivery provider discovery and booking for heavy goods.
- Quotation, negotiation, and messaging.
- Order tracking and job tracking.
- Multi-party reviews.
- Wallet, invoices, and transaction history.
- Notifications and support.

## Merchant modules

The merchant dashboard must include:

- Store profile and verification.
- Product listing and media management.

- Inventory, availability, and lead time management.
- Pricing and quotation handling.
- Pickup readiness and delivery preparation.
- Store order management.
- Customer communication.
- Reviews of customers and transaction histories.
- Financial settlements, payout status, and analytics.

## Technician modules

The technician dashboard must include:

- Professional profile, license data, and skills taxonomy.
- Service areas and map radius.
- Availability calendar.
- Quote inbox and negotiation tools.
- Chat and correspondence.
- Work plan, phases, milestones, and schedule creation.
- Time and cost estimation.
- Job acceptance and completion workflow.
- Earnings and payout history.
- Customer review and dispute participation.

## Delivery provider modules

The driver or fleet dashboard must include:

- Vehicle profile and capacity.
- Service zone and live availability.
- Delivery job marketplace or dispatch inbox.
- Navigation and tracking.
- Proof of pickup and proof of delivery.
- Delivery completion and timeout rules.
- Earnings, payout history, and customer reviews.

## Admin modules

The admin portal must include:

- User verification and KYC workflow.
- Product and service taxonomy management.
- Pricing rules, commission rules, and payout rules.
- Dispute management.
- Fraud and abuse monitoring.
- Content moderation.
- SLA monitoring.
- Reports and platform analytics.

## End-to-end customer journey

## Search and discovery

The customer enters keywords into the search bar, such as product name, brand, material type, finish, color, or use case. The platform must return relevant results from the product index and backend catalog, displayed as matrix grid cards with image, title, short description, vendor/store name, price or quotation badge, review score, stock/availability, delivery or pickup option, technician-available flag, and estimated fulfillment time. Advanced search and filtering are essential marketplace features for building materials, especially by specification, availability, quantity, and delivery readiness.coaxsoft

The search results must support sorting and filtering by:

- Relevance.
- Price low to high / high to low.
- Nearest store.
- Popularity.
- Highest review score.
- Availability of technician.
- Delivery time.
- Stock availability.
- Brand.
- Material specification.

- Quantity or pack size.
- Pickup availability.
- Quote-only items.

## Product selection

From search results, the customer can:

- Open product detail page.
- Add item to cart.
- Buy now.
- View store location.
- Choose pickup or delivery.
- Indicate whether technician help is needed.
- View “available now” or lead time such as “in 5 days.”
- Request quotation if no fixed price exists.

For construction categories, product pages must also support technical specifications, quantity rules, bulk ordering support, and delivery suitability notes.coaxsoft

## Cart and technician continuation

If the customer adds the item and selects “I need technician to fix it,” the cart icon must show a live item notification while the journey continues to a technician selection page. This page should list plumbers, painters, electricians, mechanical technicians, and other specialists with cards showing completed jobs, hourly rate or fixed cost or quotation button, review score, current availability, current location, ETA to customer location, service radius, and “Hire now.” Marketplace systems often require branch- or worker-availability logic and employee timetable management, which supports this design pattern.creative-sols

By default, technicians should be ranked by:

1. Distance to customer.
2. Affiliation with the merchant from whom the material was purchased, if applicable.
3. Availability.
4. Review score.
5. Relevant skill match.

The customer must also be able to sort technicians by cost, hourly rate, availability, review score, location, ETA, and merchant affiliation.

## Quotation and negotiation

If the customer presses “Get quotation,” the system must immediately generate a structured request message to the worker mailbox and in-app notifications. The default request must describe:

- Purchased material.
- Intended job type.
- Worksite address or approximate zone.
- Requested schedule.
- Photos or notes if provided.
- Whether measurement visit is required.

The worker can then press “Contact client for more details,” which opens secure platform chat and optional email relay. After discussion, the worker submits:

- Work plan.
- Work phases.
- Time schedule.
- Cost per phase or total cost.
- Materials included or excluded.
- Validity period of quotation.
- Start date and expected completion.

The customer can agree, negotiate, or decline. If agreed, the worker is marked as hired and the job moves into active status. This structure mirrors marketplace requirements for support channels, technical clarification, and operational workflow mapping across all roles.coaxsoft

## Checkout and payment

At any time, the customer can return to checkout from the cart icon. The cart page must allow:

- Increase quantity.

- Decrease quantity.
- Remove item.
- Save for later.
- Review pickup/delivery choice.
- Review technician booking state.
- Review delivery booking state.
- View subtotal per component.

The payment page must support, in phase 1:

- Pay on platform by card.
- Pay at store / pay on site for pickup orders where this is allowed.

Later phases can add:

- Apple Pay.
- Google Pay.
- Bank transfer.
- Buy now, pay later.
- Wallet.
- Corporate credit account.
- Cash on delivery where legally and operationally feasible.

Construction commerce often needs robust payment methods, purchase orders, credit terms, and escrow-like controls for large transactions, although these can be later-phase capabilities.coaxsoft

## Delivery logic

## Pickup

If the customer chooses pickup and pay-on-site, the order can complete as a confirmed pickup reservation without card capture if merchant policy allows it. The merchant receives order notification, prepares the item, and marks it ready for pickup.coaxsoft

If the customer chooses pickup and card payment, the user enters card details, may save the card for later, receives payment confirmation, and the merchant is notified to prepare the item. After pickup is confirmed, the review sequence begins.

## Small-item delivery

For small and light goods, the customer can choose direct delivery. After checkout, the tracking page should display live driver progress on map in a familiar courier-style format, with order status, assigned driver, ETA, route progress, and support options. Real-time delivery tracking is consistent with marketplace order tracking requirements from purchase through fulfillment.coaxsoft

## Heavy and bulk delivery

For heavy goods such as tons of cement or rebars, the merchant must define delivery constraints on each product or SKU, including:

- Unit weight.
- Unit dimensions.
- Stackability.
- Hazard flags if any.
- Minimum vehicle type.
- Maximum safe load per vehicle type.
- Loading requirements.
- Crane or forklift requirements.
- Partial delivery rules.
- Delivery window constraints.

The system must include a vehicle suitability engine that calculates minimum vehicle specification from ordered material quantity and transport rules. Construction marketplaces require specialized shipping methods for oversized and heavy materials and often need consolidated shipping and zone-based pricing.coaxsoft

The platform must then show available delivery options such as:

- Bike.
- Small car.
- Pickup truck.
- Flatbed truck.
- Heavy truck.
- Crane truck.

Each delivery provider card should show:

- Vehicle type.
- Capacity.
- Current location.
- ETA to merchant.
- ETA to customer.
- Delivery cost.
- Review score.
- Availability.
- Special handling capability.

The customer can filter by cost, time, proximity, rating, and vehicle type before proceeding to payment and tracking.

## Reviews and closeout

## Customer review page

After the product is picked up, delivered, or job completed, the customer should be directed to a unified but role-aware review center. This page should be designed as a post-transaction quality closure workspace, not a simple star-rating form.

The page shape should include:

- Top progress banner: “Order completed,” “Delivery completed,” “Service completed,” with badges for each completed component.
- Summary panel: purchased product(s), store name, technician name if any, driver name if any, dates, and total paid split by category.
- Review tabs or stacked cards: “Review product,” “Review store,” “Review technician,” “Review driver.”
- For product review card: star rating, quality rating, value rating, accuracy vs description, media upload, free-text comments, “would buy again.”
- For store review card: communication, packaging/preparation, honesty of listing, pickup/delivery readiness, overall satisfaction.
- For technician review card: professionalism, punctuality, quality of work, cleanliness, communication, adherence to quotation, would hire again.

- For driver review card: punctuality, care of material, communication, navigation reliability, delivery professionalism.
- Final action buttons: submit all, submit later, report issue.

This page should also support issue escalation links such as “item damaged,” “worker no-show,” or “delivery delayed.”

## Merchant review-of-customer page

The merchant dashboard should include a structured customer evaluation page triggered after order closure. This page should allow the merchant to assess the customer’s transaction behavior in a controlled and legally safe manner.

The page shape should include:

- Header with order reference, customer masked identity, and fulfillment type.
- Behavior assessment chips: responsive, punctual pickup, payment reliability, communication quality, no-show risk, dispute tendency.
- Transaction integrity section: whether customer respected pickup window, whether complaints were justified, whether return behavior was normal.
- Free-text internal note area visible only to merchant/admin, subject to moderation policy.
- Recommendation field: “Would serve again.”
- Submission lock after a set review window.

A similar closure review capability should exist for technician and driver dashboards. Because the platform is multi-sided, each participant needs transaction history and retrospective evaluation capability.sharetribe+1

## Upwork-like technician workflow

If the customer hires a technician, the service execution module should follow a milestone-capable workflow similar in principle to freelance job platforms:

- Quote accepted.
- Work order created.
- Schedule confirmed.
- Optional deposit paid.

- Phase 1 in progress.
- Phase 1 approved or disputed.
- Next phase released.
- Final completion submitted.
- Customer confirms completion.
- Review and payout release.

This model is especially useful for painting, installation, repair, and multi-step maintenance jobs, where scope, timing, and phased payment matter.

## Payment orchestration and settlement

The platform must support a single customer checkout that computes:

- Material subtotal.
- Technician subtotal.
- Delivery subtotal.
- Platform commission(s).
- Tax/VAT if applicable.
- Discounts or coupons.
- Refundable and non-refundable components.

The platform then allocates funds to the relevant payees separately:

- Material proceeds to merchant bank account.
- Job proceeds to technician bank account.
- Delivery proceeds to driver or vehicle owner bank account.
- Platform commission retained by marketplace.

Marketplace payment systems such as Stripe Connect support split payments, embedded onboarding, identity verification, delayed payouts, and automated fee deduction, making them suitable for this requirement.stripe+1

The system should support these payout states:

- Pending.
- On hold.
- Released.
- Failed.

- Reversed.
- Refunded partially.
- Refunded fully.
- Disputed.

Funds release policy should be configurable per transaction type. For example, merchant funds may release after pickup confirmation or delivery proof, technician funds after customer sign-off or milestone approval, and driver funds after proof of delivery. Delayed payout control is a standard marketplace payment feature.[sharetribe]

## Dashboard transaction history

Each actor must have a financial history page showing:

- Transaction date.
- Order or job reference.
- Gross amount.
- Platform fee.
- Net amount.
- Status.
- Payout reference.
- Linked documents or invoice.
- Counterparty.
- Notes.
- Dispute or refund flag.

This history must appear in:

- Customer dashboard.
- Merchant dashboard.
- Technician dashboard.
- Driver dashboard.
- Admin reporting.

The platform should also include an analytics screen for each role. Construction marketplaces are expected to integrate financial reporting and operational visibility across multiple vendors and locations.[coaxsoft]

# Analytics screens

## Customer analytics

Include:

- Total purchases.
- Total jobs booked.
- Total spent by category.
- Favorite stores.
- Favorite technicians.
- Delivery usage rate.
- Open issues.
- Repeat purchase intervals.

## Merchant analytics

Include:

- Revenue.
- Orders by status.
- Top products.
- Quote conversion rate.
- Pickup vs delivery split.
- Average fulfillment time.
- Cancellation rate.
- Review score trend.
- Payout summary.

## Technician analytics

Include:

- Jobs won.
- Quote acceptance rate.
- Revenue by service type.

- Utilization rate.
- Average response time.
- Completion time.
- Review score trend.
- Repeat customers.
- Payout summary.

## Driver analytics

Include:

- Deliveries completed.
- Revenue.
- On-time delivery rate.
- Average route time.
- Capacity utilization.
- Rating trend.
- Failed delivery rate.
- Payout summary.

## Admin analytics

Include:

- GMV.
- Take rate.
- Active merchants.
- Active technicians.
- Active drivers.
- Conversion funnel.
- Quote volume.
- Dispute rate.
- Refund rate.
- Settlement latency.

# Information architecture

## Customer web and mobile IA

- Home
- Search results
- Product details
- Store profile
- Cart
- Delivery options
- Technician options
- Checkout
- Payment
- Order tracking
- Messages
- Reviews
- Profile
- Addresses
- Payment methods
- Order history
- Project history
- Support

## Merchant IA

- Dashboard
- Orders
- Products
- Inventory
- Quotes
- Delivery prep
- Customers
- Reviews
- Finance
- Payouts

- Analytics
- Settings

## Technician IA

- Dashboard
- Leads / quote requests
- Messages
- Schedule
- Jobs
- Milestones
- Earnings
- Reviews
- Profile
- Settings

## Driver IA

- Dashboard
- Available jobs
- Assigned deliveries
- Navigation / tracking
- Proof of delivery
- Earnings
- Reviews
- Vehicle profile
- Settings

## Admin IA

- Overview
- Users
- Merchants
- Workers
- Drivers
- Products

- Categories
- Orders
- Jobs
- Deliveries
- Payments
- Payouts
- Disputes
- Moderation
- Analytics
- Configuration

## Search requirements

The search system must support:

- Full-text keyword search.
- Synonym mapping, such as “wash sink” and “basin.”
- Brand search.
- Arabic and English support.
- Typo tolerance.
- Attribute-based faceting.
- Geo-aware ranking.
- Availability-aware ranking.
- Quote-only and price-known distinctions.

Relevant search and faceted filtering are essential for e-commerce and specifically emphasized for building materials marketplaces.coaxsoft

## Data model

At minimum, the backend domain model should contain:

- User
- Role
- Address
- GeoPoint
- Merchant

- StoreBranch
- Product
- ProductVariant
- ProductSpecification
- InventoryRecord
- Price
- QuoteRequest
- QuoteResponse
- Cart
- CartItem
- Order
- OrderLine
- DeliveryOption
- VehicleType
- DeliveryJob
- TechnicianProfile
- Skill
- ServiceArea
- WorkOrder
- WorkPhase
- Milestone
- ChatThread
- Message
- Review
- RatingDimension
- PaymentIntent
- SplitAllocation
- Payout
- Refund
- Dispute
- Notification
- AuditLog

## Non-functional requirements

The platform should meet these non-functional requirements:

- Real-time notifications for quote, chat, status, and payout events.
- High search responsiveness.
- Multi-language capability, especially Arabic and English.
- Mobile-first workflows.
- Role-based access control.
- Auditability for all money movement.
- High availability for order and payment services.
- Data privacy and encryption.
- Scalable catalog and search indexing.
- Observability, tracing, and fraud logging.

Construction marketplaces also need interoperability, robust infrastructure planning, and integration with ERP, inventory, and financial systems to avoid stock and pricing inconsistency.shopify+1

## Integrations

Phase 1 or phase 2 integrations should include:

- Payment orchestration provider such as Stripe Connect for marketplace splits and seller onboarding.stripe+1
- Maps and geolocation provider for saved addresses, drag-and-drop pinning, distance calculations, ETA, and live tracking.
- Push notifications and email.
- SMS or WhatsApp notifications where lawful and desired.
- ERP / POS / inventory systems for merchants.
- Tax and invoicing systems.
- Identity verification / KYC provider for merchants, technicians, and drivers.
- File storage for product images, licenses, invoices, and proof-of-delivery files.

## Compliance and trust

The platform must include:

- Merchant verification.
- Worker license verification where relevant.
- Driver and vehicle document verification.

- Terms for review moderation and anti-defamation controls.
- Building-code or safety document support where certain products require regulated data.
- Tax-compliant invoicing and transaction records.

Construction commerce guidance specifically highlights regulatory compliance, documentation, and professional licensing verification for restricted products or services.coaxsoft

## Recommended architecture

At system level, I recommend:

- Front end: web app plus shared-design mobile apps.
- API gateway and auth service.
- Catalog service.
- Search service.
- Cart and checkout service.
- Order orchestration service.
- Quotation and negotiation service.
- Technician service marketplace.
- Delivery dispatch and tracking service.
- Payments and payout service.
- Review and reputation service.
- Notification service.
- Reporting and analytics service.

Because your platform is multi-sided and payout-sensitive, the most important architectural principle is separation of transactional domains with one orchestration layer above them. That is the cleanest way to support one user payment and many beneficiary settlements.stripe+1

## MVP definition

A strong MVP should include:

- Customer registration and saved addresses.
- Merchant onboarding.

- Product search and faceted filters.
- Product detail pages.
- Cart and checkout.
- Card payment plus pickup pay-on-site option.
- Merchant order dashboard.
- Technician marketplace with quote request and chat.
- Simple delivery selection for small items.
- Review center.
- Split-settlement ready payment architecture.
- Role-specific transaction history.

Bulk logistics optimization, advanced milestone escrow, enterprise credit terms, and full freight dispatch optimization can be phase 2 or phase 3. This phased approach matches construction marketplace guidance that recommends defining key operational functionality first, then expanding integrations and complexity as the business scales.coaxsoft

## Phase roadmap

## Phase 1

- Marketplace foundation.
- Search and catalog.
- Cart and checkout.
- Merchant dashboard.
- Technician quote and hire flow.
- Small-item delivery.
- Basic payout splitting.
- Reviews and notifications.

## Phase 2

- Bulk materials logistics engine.
- Vehicle suitability calculator.
- Driver marketplace.
- Milestone-based technician payments.

- More payment methods.
- Better analytics.
- ERP integrations.

## Phase 3

- Escrow-like staged payments.
- Enterprise contractor accounts.
- Purchase orders and credit terms.
- AI-assisted problem diagnosis and guided procurement.
- Dynamic dispatch optimization.
- Predictive pricing and demand planning.

## Product design guidance

The UI should not be designed as a generic retail fashion store. It should use compact, dense, role-based web application patterns with clear labels, visible state, fast checkout, faceted search, and dashboard-style management views, because e-commerce and data-dense marketplaces perform best when navigation architecture is explicit and operational status is always visible.coaxsoft

For the customer experience, the homepage should emphasize “solve a problem” journeys in addition to category navigation, such as “Fix leak,” “Paint room,” “Install sink,” and “Order bulk cement.” This would align the platform with the real user intent you described, where the customer often begins from a maintenance problem rather than a known SKU.coaxsoft

If you want, I can next convert this into one of the following:

- a formal Software Requirements Specification document,
- a full PRD with epics and user stories,
- a database schema and ERD,
- a complete UI sitemap and screen-by-screen specification,
- or a phased technical architecture for implementation.